

Week 1

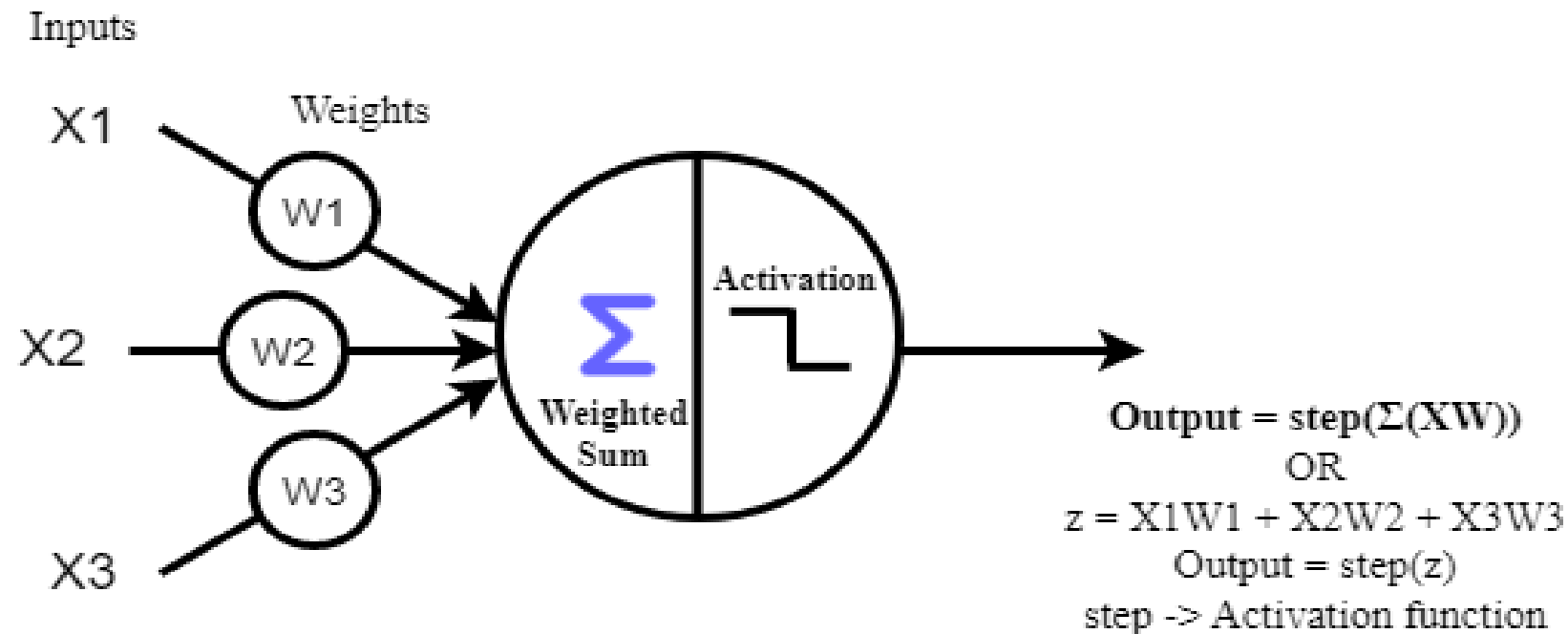
Perceptron Learning Implementation

What is Perceptron and how it has been developed?

- ❖ The word Perceptron is first ever coined in **1943** by **Warren McCulloch and Walter Pitts**. **Lately, Frank Rosenblatt**, an American psychologist, and computer scientist built the first Perceptron machine when doing research at **Cornell Aeronautical Laboratory** for performing image recognition.
- ❖ During this time the Perceptron machine become popular among the AI community and is considered the fundamental part of building intelligent systems.
- ❖ **The perceptron algorithm is based on the concept of a single neuron in the human brain, a single neuron in the human brain is doing a very simple thing, just receiving some inputs and if the inputs are high, it activates and send the signal to the next neuron.**
- ❖ The perceptron was designed to **mimic (copycat) this process**, with the input data serving as the input to the neuron and the weights representing the strength of the connections between the input neurons and the output neuron.

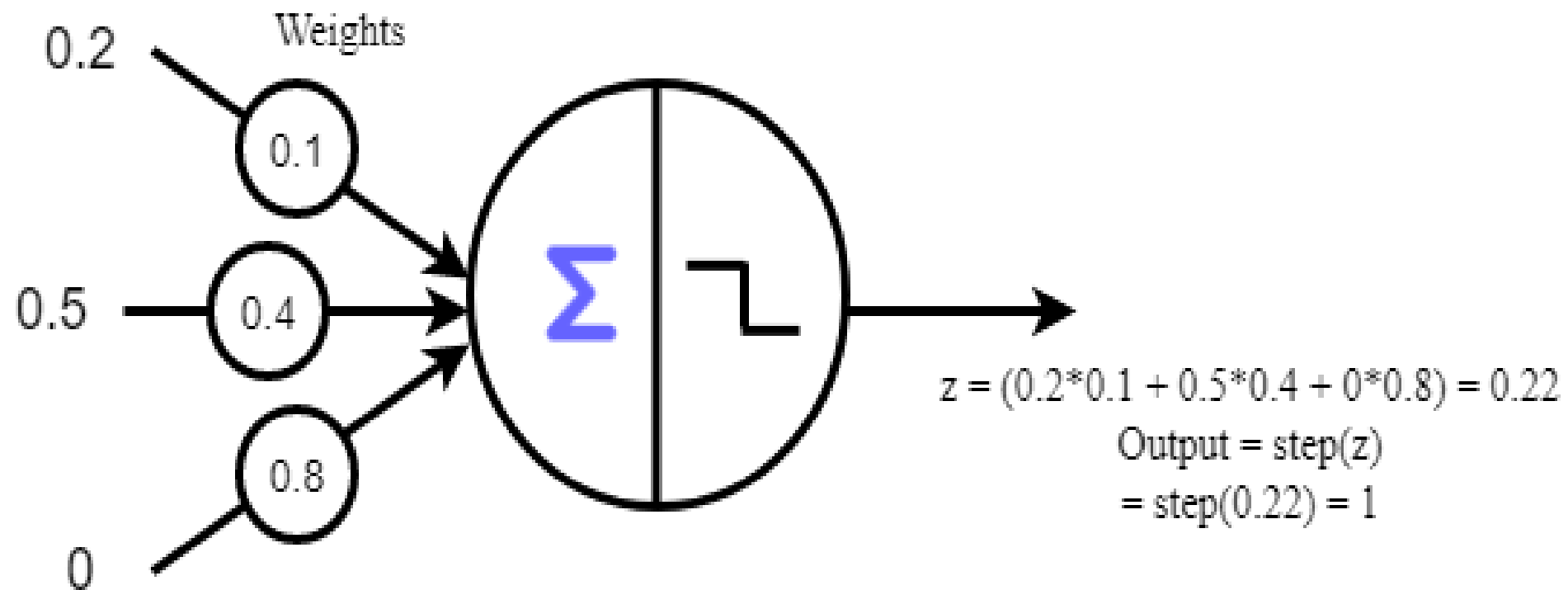
The Perceptron Algorithm, How does it work?

- ❖ The Perceptron is a type of linear classifier (binary classifier), which means it can be used to classify data that is linearly separable.
- ❖ A Perceptron is somehow similar to Logistic Regression at first glance, but it's different. While Logistic Regression predicts probabilities of a data point falling in a particular class, Perceptron will only tell whether the data point is in a particular class or not, Just like saying "Yes" or "No". Here is the diagrammatic representation of the perceptron algorithm.



- ❖ A Perceptron is a kind of single Artificial Neuron which is also known as a Threshold Processing Unit / Threshold Logic Unit (TLU).
- ❖ As you can see in the above diagram, the Perceptron contains some input links X_1 , X_2 , and, X_3 .
- ❖ Each input has its own corresponding weights W_1 , W_2 , and W_3 . These weights are basically the hearts of Perceptrons which determine the strength of each input signal to it.
- ❖ The Perceptron or Threshold Logic Unit (TLU) computes the weighted sum of the inputs ($z = X_1W_1 + X_2W_2 + X_3W_3 \dots + X_nW_n$) and then these weighted sums of inputs are passed through an activation function also known as the step function.
- ❖ These activation functions will determine whether the Perceptron needs to be activated or not.
- ❖ Let's see an example to understand this,

Inputs



Heaviside function

- ❖ The Heaviside activation function will return 0 when the weighted sum of inputs is less than zero and return 1 if it is greater than or equal to 0.]

$$\text{heaviside}(z) = \left\{ \begin{array}{ll} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{array} \right\}$$

Sign function

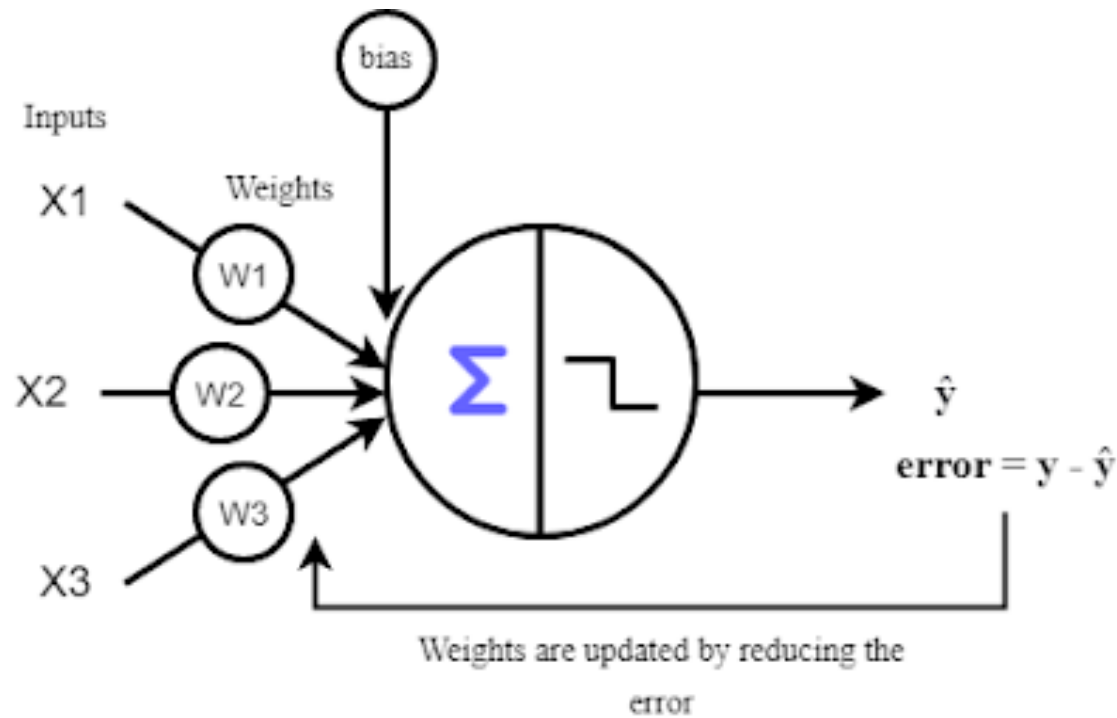
- ❖ The Sign function will return 0 if the weighted sum of inputs is 0 and return +1 and -1 when the weighted sum of inputs is greater and lesser than 0 respectively.

$$\text{sgn}(z) = \left\{ \begin{array}{lll} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{array} \right\}$$

Learning in Perceptron

- ❖ The whole point of training the perceptron is to adjust the weights of each input according to the training data.
- ❖ For training the perceptron to find the best weights, Frank Rosenblatt proposed an algorithm that is largely inspired by Hebb's rule.
- ❖ **Hebb's rule state that when a biological neuron triggers another neuron, the connection between these two neurons grows stronger.**
- ❖ There is an awesome phrase for this property of neurons, ie, **Neurons that fire together wire together.**
- ❖ Inspired by Hebb's rule, the Perceptron can be trained by finding the error made by the algorithm and finding the weights that reduce this error as much as possible.
- ❖ This is the main goal of training a machine learning algorithm in general.

- ❖ The algorithm will first make a prediction based on the training data given, then this prediction is compared with the actual desired output.
- ❖ The error is calculated for each of the training instances by comparing the prediction and actual output, finally, this error is reduced during the training process and the weights corresponding to the correct predictions are adjusted accordingly.



y = Actual values
 $\hat{y}$ = Prediction

❖ The learning(training) rule can be written as

$$w_{i,j}(next) = w_{i,j} + \eta(y_i - \hat{y})x_i$$

- ❖ Where $w_{i,j}$ is the weights between i th input neuron and j th output neuron
 - ❖ **η (eta) is the learning rate**
 - ❖ y is the target output or the actual values
 - ❖ $\hat{y}$ is the predicted value of the output neuron
 - ❖ x_i is the input value of the current training instance
- ❖ The algorithm is being trained by updating the weights until getting the optimal results.
- ❖ The learning rate(η) is required to determine the length of each step taken for finding optimal weights. Sometimes, it is better to have a smaller learning rate to get a minimum error, but sometimes a large learning rate is also preferred.

Single Layer Perceptron Model:

- ❖ This is one of the easiest Artificial neural networks (ANN) types.
- ❖ A single-layered perceptron model consists feed-forward network and also includes a threshold transfer function inside the model.
- ❖ The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.
- ❖ In a single layer perceptron model, its algorithms do not contain recorded data, so it begins with inconstantly allocated input for weight parameters.
- ❖ Further, it sums up all inputs (weight). After adding all inputs, if the total sum of all inputs is more than a pre-determined value, the model gets activated and shows the output value as +1.

Implementing Perceptron in Python

Aim:

- ❖ To implement a Single-Layer Perceptron model using TensorFlow to classify Setosa and Non-Setosa species from the Iris dataset, by applying data preprocessing, training the model, evaluating its performance using a classification report, and visualizing the decision boundary.

epoch

- ❖ In deep learning, an epoch is a single pass of all training data through an algorithm.
- ❖ It's a hyperparameter that determines the training process for a machine learning model.
- ❖ During an epoch, the training data is broken down into smaller batches, called batches, and fed into the model one at a time.
- ❖ An epoch is one complete pass through the entire training dataset by the learning algorithm.
- ❖ During an epoch, all the training samples (data points) are fed into the model once.
- ❖ This means the model sees every sample once and uses it to update the weights using backpropagation and gradient descent.
- ❖ The number of epochs specifies how long the network will be trained, but not in terms of time.
- ❖ The number of epochs is a hyperparameter that defines the number times that the learning algorithm will work through the entire training dataset

❖ Example to Understand

❖ **`history = model.fit(X_train, y_train, epochs=10, batch_size=100, verbose=1)`**

❖ Suppose we have:

- ❖ 1,000 training images
- ❖ A batch size of 100 (i.e., 100 samples are processed at a time)
- ❖ 10 epochs during training

❖ Then:

- ❖ 1 Epoch = 10 batches ($100 \times 10 = 1,000$ samples)
- ❖ 10 Epochs = The model sees all 1,000 images 10 times in total.

❖ Neural networks do not learn enough in just one pass of the data.

❖ They need to see the data many times to:

- ❖ Adjust the weights gradually
- ❖ Reduce the loss function
- ❖ Improve accuracy and generalize better on unseen data

❖ **What Happens in Each Epoch?**

- ❖ **Forward Pass:** Input data is passed through the model to get predictions.
- ❖ **Loss Calculation:** The prediction is compared with the actual output using a loss function.
- ❖ **Backward Pass:** The model calculates gradients (errors).
- ❖ **Weights Update:** The model updates the weights to reduce error using an optimizer (e.g., SGD, Adam).
- ❖ **Repeat:** This process is done for all batches in that epoch.

❖ **Impact of Number of Epochs**

❖ **Too Few Epochs:**

- ❖ The model may underfit (doesn't learn enough).
- ❖ Training and validation accuracy will be low.

❖ **Too Many Epochs:**

- ❖ The model may overfit (learns training data too well but performs poorly on new data).
- ❖ Training accuracy is high, but validation accuracy starts to drop.

❖ **model.fit(X_train, y_train, epochs=100, batch_size=32)**

This means:

- ❖ The model will go through the entire training data 100 times
- ❖ Each time, it processes 32 samples per batch

General Format

Epoch X/Y

Batches [====] - time - loss: value - accuracy: value

Where:

- ❖ X is the current epoch number.
- ❖ Y is the total number of epochs set for training.
- ❖ loss tells how far the model's predictions are from actual results.
- ❖ accuracy shows how well the model is predicting (especially in classification problems).
- ❖ The progress bar ([====]) shows how many batches have been processed.

Epoch 1/100

4/4 [=====] - 0s 6ms/step - loss: 0.1589 - accuracy: 1.0000

- ❖ **Epoch 1/100:** This is the 1st iteration out of 100 epochs.
- ❖ **4/4:** There are 4 batches in your training dataset.
 - ❖ This means your dataset is split into 4 mini-batches based on the batch size.
- ❖ **0s 6ms/step:** Time taken to process each step/batch (in milliseconds).
- ❖ **loss: 0.1589:** This is the calculated error (difference between prediction and truth).
- ❖ **accuracy: 1.0000:** This means 100% accuracy on the training set for this epoch (perfect prediction on all samples in this batch)

verbose

- ❖ In machine learning, the "verbose" parameter controls the amount of output or information displayed during training or other processes.
- ❖ It's a way to monitor the progress and behavior of your model, with higher verbosity levels providing more detailed information.
- ❖ **Levels:**
- ❖ The verbosity level is typically controlled by an integer value, often with 0, 1, and 2 as common options.
 - ❖ **verbose=0** usually means no output is generated, keeping the process silent.
 - ❖ **verbose=1** provides basic output, like a progress bar during training.
 - ❖ **verbose=2** offers more detailed output, potentially including information about each epoch or training step.

Program:

1. **#pip install tensorflow**
2. **import numpy as np**
3. **import matplotlib.pyplot as plt**
4. **import tensorflow as tf**
5. **from sklearn.datasets import load_iris**
6. **from sklearn.model_selection import train_test_split**
7. **from sklearn.preprocessing import StandardScaler**
8. **from sklearn.metrics import classification_report**

❖ This code imports essential libraries for building and evaluating a Deep learning model using the Iris dataset, including TensorFlow for model creation, NumPy and Matplotlib for data handling and visualization, and Scikit-learn for data loading, preprocessing, splitting, and performance evaluation.

- ❖ The Iris dataset is a classic dataset in machine learning, often used for introductory classification tasks. It contains measurements of iris flowers from three different species: setosa, versicolor, and virginica.
- ❖ Each sample in the dataset includes four features: sepal length, sepal width, petal length, and petal width, all measured in centimeters. The dataset consists of 150 instances, with 50 instances for each of the three iris species.
- ❖ The Iris dataset was introduced by Ronald Fisher in 1936 as an example of linear discriminant analysis.
- ❖ **The dataset contains four numerical features:**
 - ❖ Sepal length (cm) - Sepal width (cm)
 - ❖ Petal length (cm) - Petal width (cm)
- ❖ Classes:
- ❖ **It includes three classes (species) of iris flowers:**
 - ❖ Iris setosa - Iris versicolor
 - ❖ Iris virginica

iris setosa



petal

sepal

iris versicolor



petal

sepal

iris virginica



petal

sepal

1. **# Load Iris dataset**
2. **iris = load_iris()**
3. **X = iris.data**
4. **y = (iris.target == 0).astype(int) # Setosa vs non-Setosa**

❖ This code loads the Iris dataset and converts the target labels into a binary classification problem where class 0 (Setosa) is labeled as 1 and all other classes as 0.

1. **# Split the dataset into training and testing sets**
2. **X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)**

❖ This code splits the data into 70% training and 30% testing sets, with random_state=42 ensuring reproducibility by using a fixed seed for random shuffling.

1. **# Standardize the features**
2. **scaler = StandardScaler()**
3. **X_train = scaler.fit_transform(X_train)**
4. **X_test = scaler.transform(X_test)**

- ❖ The code standardizes the training and test feature data to have zero mean and unit variance using StandardScaler.
- ❖ This code standardizes the input features so that they have a mean of 0 and standard deviation of 1, which helps improve the performance and convergence speed of many machine learning algorithms.
- ❖ Ensures all features are on the same scale.
- ❖ Prevents features with larger values from dominating

```
1. # Build the single-layer Perceptron model using TensorFlow
2. model = tf.keras.Sequential([
3.     tf.keras.layers.Dense(1, activation='sigmoid', input_shape=(X_train.shape[1],))
4. ])
```

- ❖ This code builds a single-layer Perceptron model using TensorFlow's Keras API.
- ❖ The Sequential model represents a linear stack of layers, and here it contains **one Dense** (fully connected) layer with a single neuron (units=1) and a sigmoid activation function, which is typically used for binary classification tasks.
- ❖ The input_shape=(X_train.shape[1],) specifies the number of features in the input data, ensuring the model knows the shape of each input sample.
- ❖ This simple model will output a probability between 0 and 1 for binary classification

1. **# Compile the model**

2. **model.compile(optimizer='sgd', loss='binary_crossentropy', metrics=['accuracy'])**

- ❖ This code compiles the previously defined model using the stochastic gradient descent (sgd) optimizer, sets the loss function to binary_crossentropy which is suitable for binary classification problems, and specifies accuracy as the metric to monitor during training, allowing evaluation of how well the model is learning to classify the data.

1. **# Train the model**

2. **history = model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=1)**

- ❖ This code trains the compiled model on the training data (X_train and y_train) for 100 epochs using a batch size of 32, meaning the model updates its weights after processing every 32 samples; the verbose=1 setting displays a progress bar and training details for each epoch, and the training history (like loss and accuracy per epoch) is stored in the history variable for later analysis or visualization.

```
: # Train the model
history = model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=1)
```

Epoch 1/100

4/4 [=====] - 0s 6ms/step - loss: 0.1589 - accuracy: 1.0000

Epoch 2/100

4/4 [=====] - 0s 4ms/step - loss: 0.1579 - accuracy: 1.0000

Epoch 3/100

4/4 [=====] - 0s 6ms/step - loss: 0.1570 - accuracy: 1.0000

Epoch 4/100

4/4 [=====] - 0s 6ms/step - loss: 0.1560 - accuracy: 1.0000

Epoch 5/100

4/4 [=====] - 0s 6ms/step - loss: 0.1551 - accuracy: 1.0000

Epoch 6/100

4/4 [=====] - 0s 7ms/step - loss: 0.1542 - accuracy: 1.0000

Epoch 7/100

4/4 [=====] - 0s 5ms/step - loss: 0.1031 - accuracy: 1.0000

Epoch 93/100

4/4 [=====] - 0s 6ms/step - loss: 0.1027 - accuracy: 1.0000

Epoch 94/100

4/4 [=====] - 0s 6ms/step - loss: 0.1023 - accuracy: 1.0000

Epoch 95/100

4/4 [=====] - 0s 6ms/step - loss: 0.1020 - accuracy: 1.0000

Epoch 96/100

4/4 [=====] - 0s 7ms/step - loss: 0.1016 - accuracy: 1.0000

Epoch 97/100

4/4 [=====] - 0s 8ms/step - loss: 0.1012 - accuracy: 1.0000

Epoch 98/100

4/4 [=====] - 0s 6ms/step - loss: 0.1009 - accuracy: 1.0000

Epoch 99/100

4/4 [=====] - 0s 5ms/step - loss: 0.1005 - accuracy: 1.0000

Epoch 100/100

4/4 [=====] - 0s 4ms/step - loss: 0.1001 - accuracy: 1.0000

- ❖ The loss starts at 0.1589 and gradually decreases to 0.1001, indicating the model is learning and minimizing errors over time.
- ❖ The accuracy remains 1.0000 (100%) throughout training, meaning the model is correctly classifying all training examples — it's either very effective or possibly overfitting if the dataset is too small or simple.

1. # Predict on the test set

2. `predictions = model.predict(X_test)`

3. `binary_predictions = (predictions > 0.5).astype(int)`

2/2 [=====] - 0s 0s/step

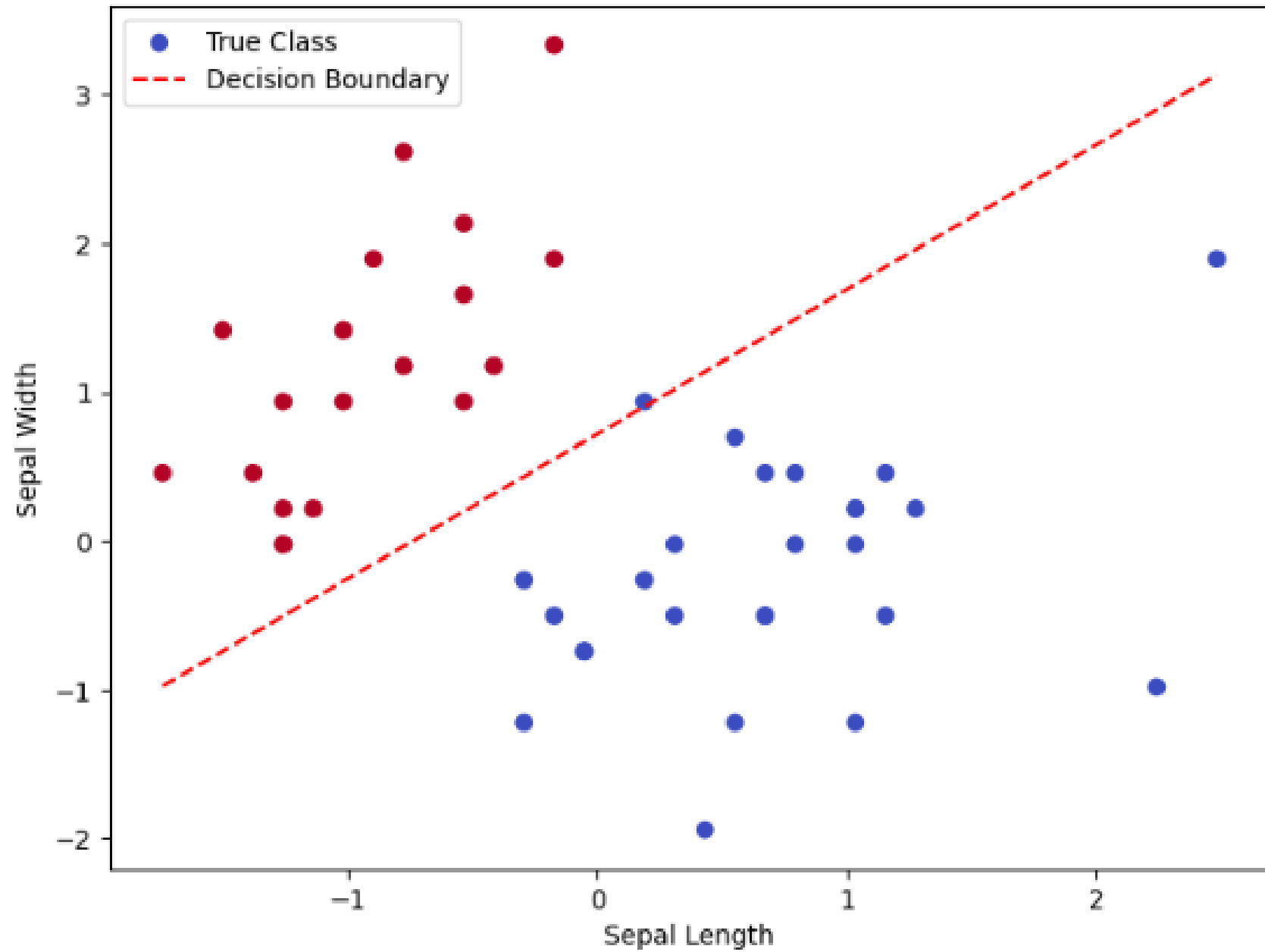
- ❖ This code makes predictions on the test dataset using the trained model; `model.predict(X_test)` outputs probability scores (between 0 and 1) for each test sample, indicating how likely they belong to the positive class. Since this is a binary classification task, the line `binary_predictions = (predictions > 0.5).astype(int)` converts these probabilities into binary class labels: values greater than 0.5 are classified as 1 (positive class), and those less than or equal to 0.5 as 0 (negative class).

1. `# Generate classification report`
2. `class_names = ['Non-Setosa', 'Setosa'] # Order corresponds to the labels`
3. `report = classification_report(y_test, binary_predictions,
target_names=class_names)`
4. `print("Classification Report:\n")`
5. `print(report)`

Classification Report:

	precision	recall	f1-score	support
Non-Setosa	1.00	1.00	1.00	26
Setosa	1.00	1.00	1.00	19
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

1. `# Plot the decision boundary`
2. `plt.figure(figsize=(8, 6))`
3. `plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap='coolwarm', marker='o', label='True Class')`
4. `plt.xlabel('Sepal Length')`
5. `plt.ylabel('Sepal Width')`
6. `x_vals = np.linspace(X_test[:, 0].min(), X_test[:, 0].max(), 100)`
7. `y_vals = (-model.weights[0][0] * x_vals - model.weights[1]) / model.weights[0][1]`
8. `plt.plot(x_vals, y_vals, 'r--', label='Decision Boundary')`
9. `plt.legend()`
10. `plt.show()`



Conclusion:

- ❖ The Perceptron model built using TensorFlow effectively classified Setosa and Non-Setosa flowers from the Iris dataset with excellent accuracy. Through proper data preprocessing, model training, and evaluation, the model demonstrated strong capability in learning linear decision boundaries for binary classification. The high classification performance and clear visualization of the decision boundary confirm that even a simple single-layer neural network can achieve reliable results when applied to well-structured datasets like Iris. This experiment highlights the practical power of neural networks in solving real-world classification problems.